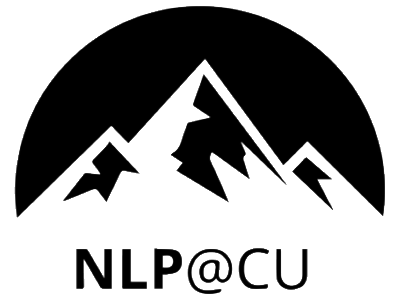


Natural Language Processing

CSCI 5832—Lecture 7

Jim Martin



Today

- Review last time
 - Loss functions
 - Derivative of loss
 - Gradient
 - Weight update
- More realistic SGD
- New HW
- More evaluation

Quiz 1

Next Thursday

- We're behind on the material. So I'm moving Ch 5 to the next quiz. This quiz will be on Chs. 1-4 only.
- I'll post a practice quiz and detailed readings later today.

Cross Entropy Loss

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

$$\begin{aligned}\log p(y|x) &= \log [\hat{y}^y (1 - \hat{y})^{1-y}] \\ &= y \log \hat{y} + (1 - y) \log (1 - \hat{y})\end{aligned}$$

$$L_{CE}(w, b) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

The negative of the log probability assigned to the correct class.

Clicker

1. A classifier has assigned a probability of 0.8 to an item belonging to the positive class. What's the CE loss?
 - a) 0.8
 - b) $-\log(0.2)$
 - c) $-\log(0.8)$
 - d) $\log(4)$

Clicker

2. A classifier has assigned a probability of 0.7 to an item belonging to the negative class. What's the CE loss?

- a) 0.7
- b) 0.3
- c) $-\log(0.3)$
- d) $-\log(0.7)$

Learning

- We want to find the weights that minimize the average loss across an entire training set.

$$\begin{aligned} \text{Cost}(w, b) &= \frac{1}{m} \sum_{i=1}^m L_{CE}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \sigma(w \cdot x^{(i)} + b) + (1 - y^{(i)}) \log (1 - \sigma(w \cdot x^{(i)} + b)) \end{aligned}$$

Learning

- We'll do this by starting with a random set of weights and then iteratively updating those weights to lower this cost.

$$\begin{aligned} \text{Cost}(w, b) &= \frac{1}{m} \sum_{i=1}^m L_{CE}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \sigma(w \cdot x^{(i)} + b) + (1 - y^{(i)}) \log (1 - \sigma(w \cdot x^{(i)} + b)) \end{aligned}$$

Learning

$$w_{t+1} = w_t - \mu \Delta$$

Derivatives

Fortunately, calculus tells us how to do that. The derivative of the loss function with respect to the weights tells us the direction and magnitude of change we should make to each weight.

Partial Derivative

- In real applications we have many features/weights not just one. So we have a partial derivative problem.
- We need the vector of the partial derivatives of the loss wrt the weights
 - Call that the gradient
 - The gradient tells us how to change the weights to minimize the loss

Partial Derivative

$$L_{CE}(w, b) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

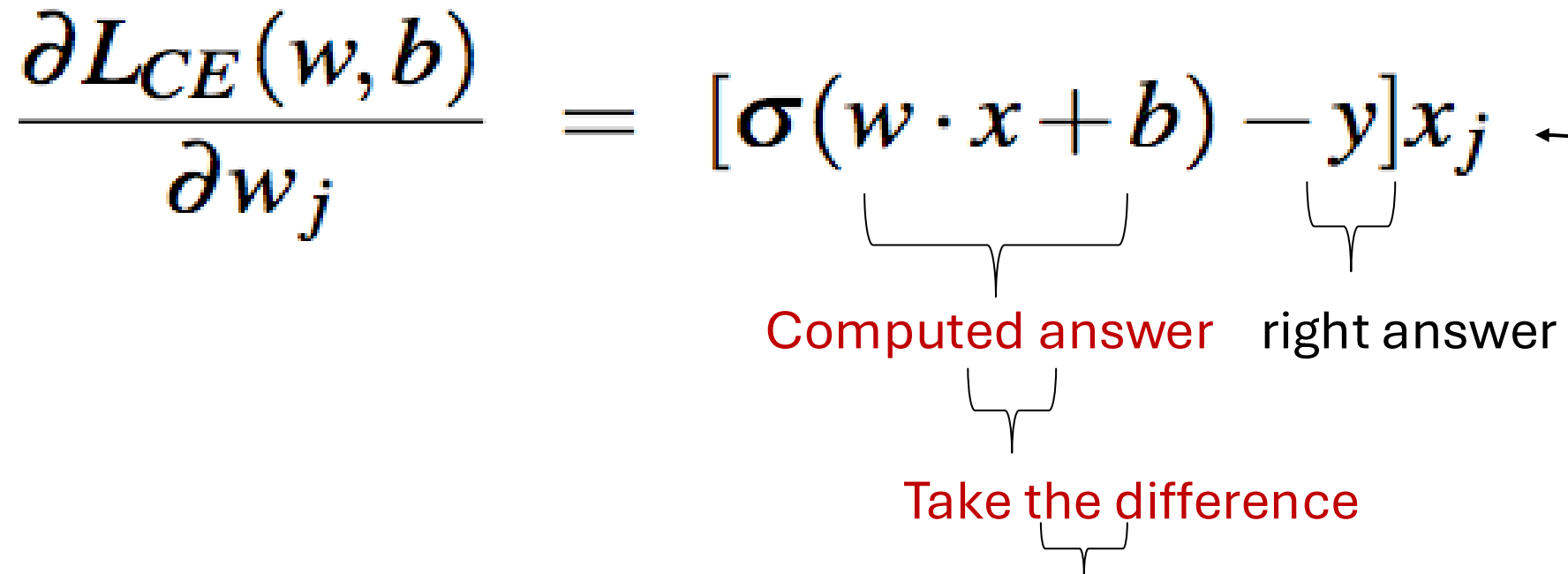
$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

If we have n weights, we end up with n partial derivatives. The vector consisting of those derivatives is called the gradient. We'll use the gradient to update all the weights.

CE Loss Partial Derivative

$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = [\underbrace{\sigma(w \cdot x + b)}_{\text{Computed answer}} - \underbrace{y}_{\text{right answer}}] x_j$$

Take the difference



Multiply by the value of the input feature for the weight being adjusted

Multiclass Models

- Multinomial logistic regression (Softmax)
- We'll still assume we have objects represented as vectors of features, and weights on the features, and a scoring function.

Softmax

$$p(y = c|x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{j=1}^k e^{w_j \cdot x + b_j}}$$

Cross Entropy Loss

$$\begin{aligned} L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) &= -\sum_{k=1}^K \mathbf{y}_k \log \hat{\mathbf{y}}_k \\ &= -\log \hat{\mathbf{y}}_c, \quad (\text{where } c \text{ is the correct class}) \\ &= -\log \hat{p}(\mathbf{y}_c = 1 | \mathbf{x}) \quad (\text{where } c \text{ is the correct class}) \\ &= -\log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \quad (c \text{ is the correct class}) \end{aligned}$$

Cross Entropy Loss

$$L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K \mathbf{y}_k \log \hat{\mathbf{y}}_k$$

$$= -\log \hat{\mathbf{y}}_c, \quad (\text{where } c \text{ is the correct class})$$

$$= -\log \hat{p}(\mathbf{y}_c = 1 | \mathbf{x}) \quad (\text{where } c \text{ is the correct class})$$

$$= -\log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \quad (c \text{ is the correct class})$$

CE Loss Partial Derivative

Looks like gobbledygook, but it matches our earlier intuition about the loss and “credit/blame” assignment for the error

$$\begin{aligned}\frac{\partial L_{\text{CE}}}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k - \hat{\mathbf{y}}_k) \mathbf{x}_i \\ &= -(\mathbf{y}_k - p(\mathbf{y}_k = 1 | \mathbf{x})) \mathbf{x}_i \\ &= -\left(\underbrace{\mathbf{y}_k}_{\text{Correct answer}} - \underbrace{\frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}}_{\text{Computed answer}} \right) \mathbf{x}_i\end{aligned}$$

Difference

Scaled by the magnitude of the feature value.

Partial Derivative

$$\begin{aligned}\frac{\partial L_{\text{CE}}}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k - \hat{\mathbf{y}}_k) \mathbf{x}_i \\ &= -(\mathbf{y}_k - p(\mathbf{y}_k = 1 | \mathbf{x})) \mathbf{x}_i \\ &= -\left(\mathbf{y}_k - \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \right) \mathbf{x}_i\end{aligned}$$

If we have k weights, we end up with k partial derivatives. The vector consisting of those derivatives is called the gradient. We'll use the gradient to update all the weights.

Practical Training

- Batch training
 - Process each example in the training set and accumulate the gradients
 - Do a single update
 - Repeat
- Minibatch training
 - Select N examples and proceed as with batch training
 - Update after each mini-batch
 - N is chosen to maximize parallelism
- Early stopping
- Matrix calculations

Data Prep

$$\mathbf{X} \in \mathbb{R}^{m \times n}$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_{11} & \mathbf{x}_{12} & \cdots & \mathbf{x}_{1n} \\ \mathbf{x}_{21} & \mathbf{x}_{22} & \cdots & \mathbf{x}_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{m1} & \mathbf{x}_{m2} & \cdots & \mathbf{x}_{mn} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

Data Prep: Standardizing Feature Values

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_{11} & \mathbf{x}_{12} & \cdots & \mathbf{x}_{1n} \\ \mathbf{x}_{21} & \mathbf{x}_{22} & \cdots & \mathbf{x}_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{m1} & \mathbf{x}_{m2} & \cdots & \mathbf{x}_{mn} \end{bmatrix}$$

$$\text{standardize}(X_{:i}) = \frac{X_{:i} - \mu_i}{\sigma_i}$$

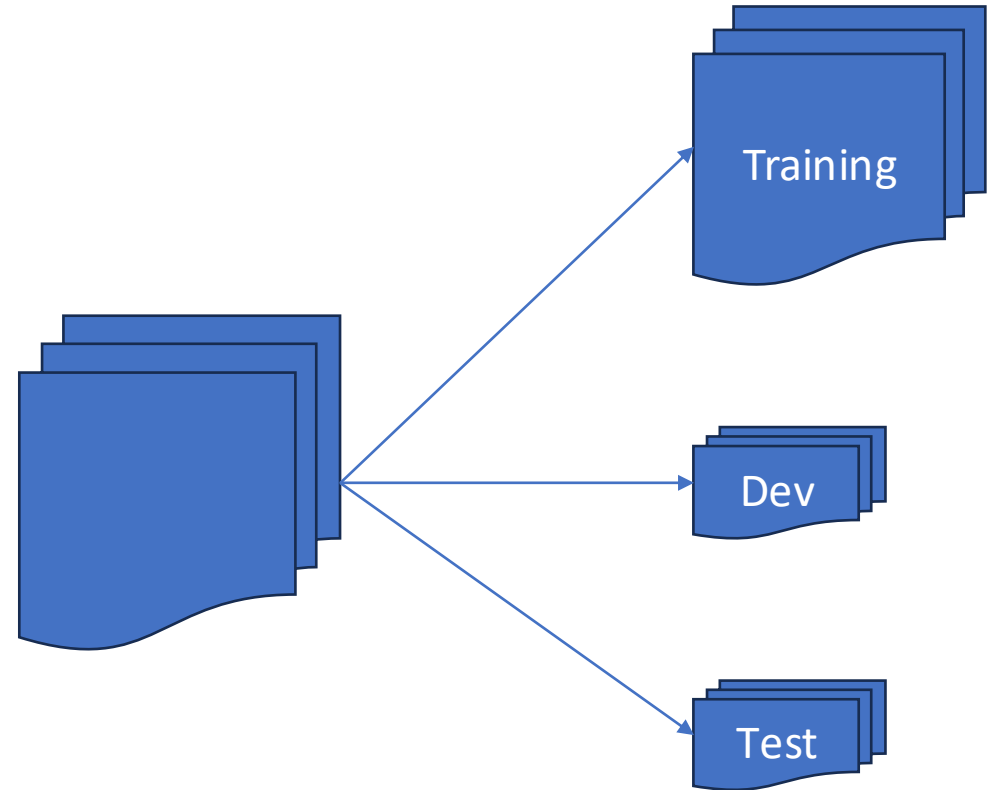
$$\mu_i = \frac{1}{m} \sum X_{:i}$$

$$\sigma_i = \sqrt{\frac{1}{m} \sum (X_{:i} - \mu_i)^2}$$

Data Preparation: Splitting the Data

In preparation for learning, we'll divide the available training data into 3 subsets

- Training
 - Examples used in training models.
- Development
 - Examples used to evaluate models under development
- Testing
 - Examples reserved only for use in final evaluation.



80/10/10 is a frequently used split.

Stochastic Gradient Descent

Now that we have an update rule, we can describe the iterative algorithm that underlies most gradient-based learning methods (including deep learning approaches).

- Given a model and a training set with N labeled samples available
 - Initialize the set of weights in the model randomly
 - In the case of logistic regression, we can use 0 for the initial values
- Repeat until happiness
 - Randomly sample an example or examples from D
 - Compute gradient
 - Use the update rule to adjust the weights

Algorithm 1 Stochastic Gradient Descent (Trad Version)

Require: $\{(x_i, y_i)\}^n$ ▷ Training set,
Require: w ▷ Model parameters,
Require: η ▷ Learning rate.

$w \leftarrow 0$

repeat

Shuffle examples in $\{(x_i, y_i)\}^n$

for $i \leftarrow 1, n$ **do**

$\nabla \leftarrow (\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i) \mathbf{x}_i$

$w \leftarrow w - \eta \nabla$

end for

until Happiness

return w

▷ Initialize weights to 0.

▷ For each epoch

▷ Example gradient

▷ Update the weights

Minibatches

- It is inefficient to update the weights based on the gradient from a single example at a time. We know the initial weights are far from where they should be and the update from a single example is likely to be idiosyncratic.
- Better to update based on a set of examples called a minibatch where the size B is $1 \ll B \ll N$.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}(w)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{1}{B} \sum_{i=1}^B [\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i] \mathbf{x}_i$$

Algorithm 2 Stochastic Gradient Descent (Minibatch Version)

Require: $\{(x_i, y_i)\}^n$ ▷ Training set,
Require: w ▷ Model parameters,
Require: m, η ▷ Batch size and learning rate.

$w \leftarrow 0$ ▷ Initialize weights to 0.

repeat

Sample a minibatch of size m from $\{(x_i, y_i)\}^n$

$\nabla \leftarrow \frac{1}{m} \sum_{i=1}^m (\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i) \mathbf{x}_i$

$w \leftarrow w - \eta \nabla$

▷ Estimate gradient.

▷ Update the weights

until Happiness

return w

Vector Processing

- The summation in this formulation implies a loop-like iteration over the examples within the mini-batch. This is an inefficient use of modern hardware.
- We can do better with matrix math and vector processing. Recall, we have all the training examples available as rows in $\mathbf{X}$ and all the answers are in the corresponding rows in $\mathbf{y}$.
- Instead of looping do a matrix multiply.

$$\frac{1}{m} \sum_{i=1}^m (\sigma(\mathbf{w}^\top \mathbf{x}_i) - \mathbf{y}_i) \mathbf{x}_i$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_{11} & \mathbf{x}_{12} & \cdots & \mathbf{x}_{1n} \\ \mathbf{x}_{21} & \mathbf{x}_{22} & \cdots & \mathbf{x}_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_{m1} & \mathbf{x}_{m2} & \cdots & \mathbf{x}_{mn} \end{bmatrix}$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

Algorithm 3 Stochastic Gradient Descent (Minibatch w/ Matrices)

Require: $\{(x_i, y_i)\}^n$ ▷ Training set,
Require: w ▷ Model parameters,
Require: m, η ▷ Batch size and learning rate.

$w \leftarrow 0$ ▷ Initialize weights to 0.
repeat
 Sample minibatch $\mathbf{X}_m$ from $\{(x_i, y_i)\}^n$
 $\nabla \leftarrow \frac{1}{m}((\sigma(\mathbf{X}_m \mathbf{w}) - \mathbf{y})^\top \mathbf{X}_m)^\top$ ▷ Estimate gradient.
 $w \leftarrow w - \eta \nabla$ ▷ Update the weights
until Happiness
return w

Let's Unpack this

$$\nabla_w \mathcal{L}_{CE} = \frac{1}{m} ((\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y})^\top \mathbf{X})^\top$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{X}\mathbf{w})$$

Compute model predictions for each item in X:

$$(m \times n) \times (n \times 1) \rightarrow (m \times 1)$$

$$\mathbf{d} = \hat{\mathbf{y}} - \mathbf{y}$$

Compute deltas for each item.

$$\mathbf{d}^\top \mathbf{X}$$

Vector of partial derivatives (gradients) for each x: $(1 \times m) \times (m \times n) = 1 \times n$

$$\frac{1}{m} (\mathbf{d}^\top \mathbf{X})^\top$$

Average over the minibatch: $(n \times 1)$

The Pursuit of Happiness

When do we stop iterating?

Optimizing logistic regression is relatively simple since the loss function is convex. Meaning that if we apply SGD with reasonable settings, we will always find the minima.

- Train epoch after epoch (minibatch after minibatch) until we reach a zero point. When the gradient is 0 and the weights will no longer change from one iteration to the next.

Practical Training: Feasibility

In practical settings, this definition of happiness may not be feasible or desirable.

We don't necessarily know a priori:

- A good batch size
- A good learning rate (or schedule)
- Whether we have enough training data.
- Whether we should be using all the features
- How to standardize the feature values

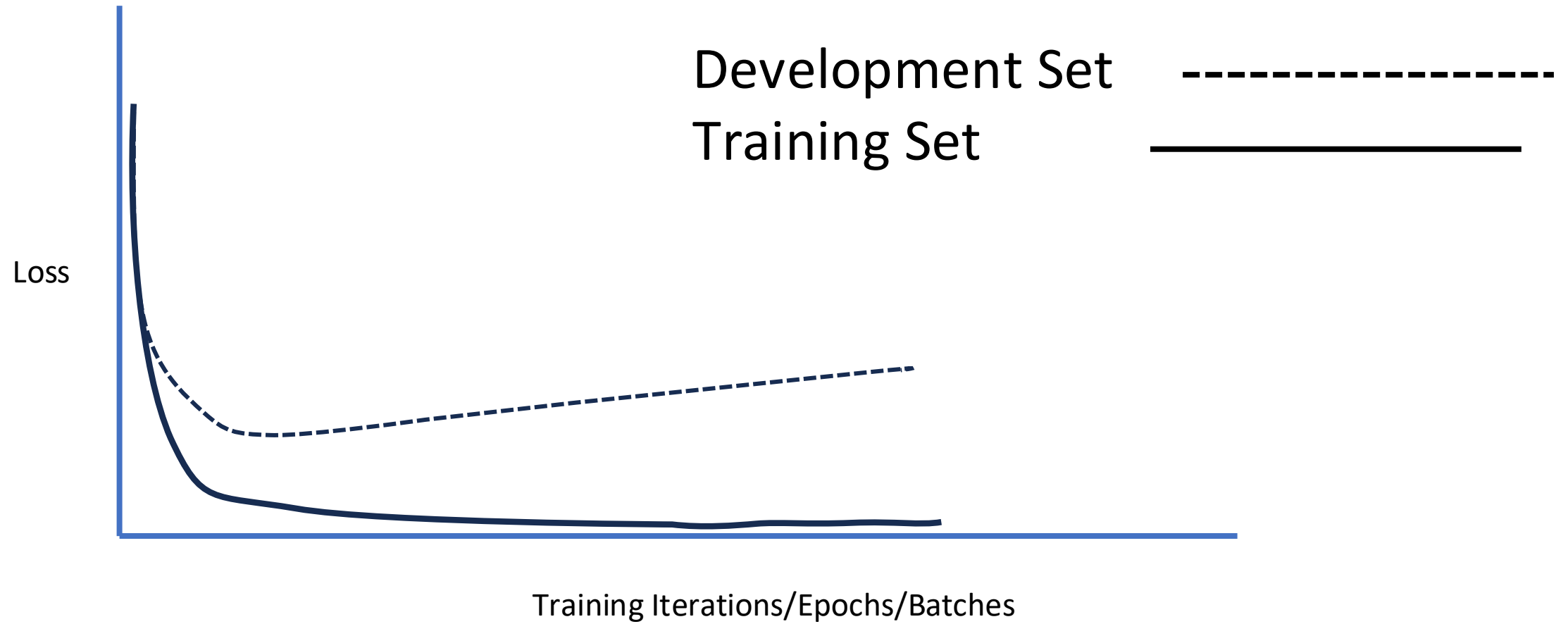
These are all matters to be discovered empirically (trial and error). And there may not be enough time or resources to explore all possibilities.

Practical Training: Desirability

In practical settings, this definition of happiness may not be feasible or desirable.

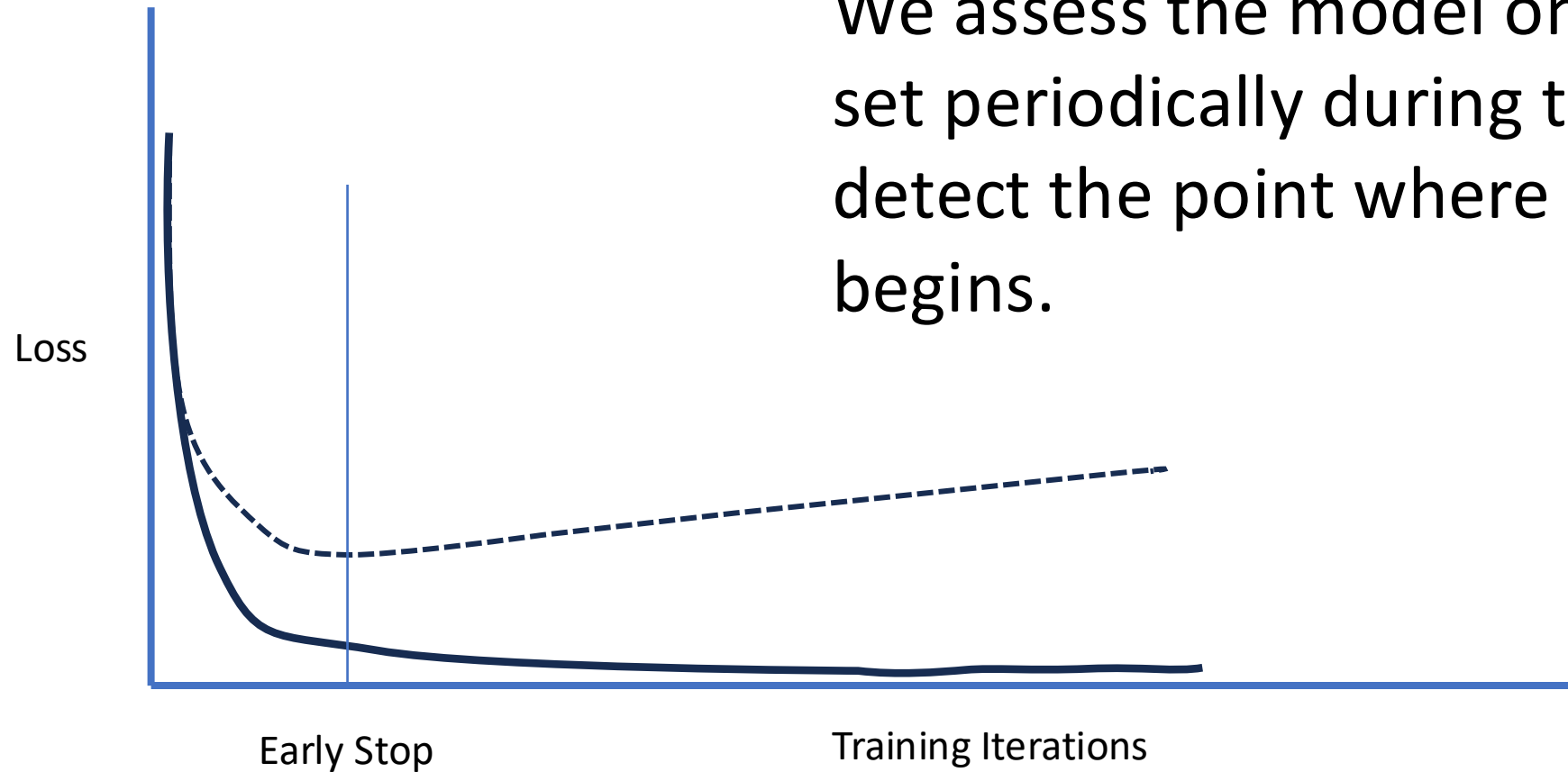
- What we care about is performance on unseen examples.
- Paradoxically, the longer we train on the training set the worse our performance may be on unseen examples!

Overfitting



Early Stopping

We assess the model on the validation set periodically during training to detect the point where overfitting begins.



Early Stopping

Patience

- Specify a period of time during which we're happy to tolerate an increase in the loss on the validation set.

Checkpoints

- Periodically, save away models that reduce the loss. We can revert to saved good models when we've overshoot and run out of patience with models that overfit.

HW 2